

IMPLEMENT THE FASTAPI BACKEND

Detailed 20-Page Project Development and Documentation Report

Includes: Brainstorming & Ideation • Requirement Analysis • Project Design • Project Planning • Development • Testing • Documentation • Demonstration • Diagrams • Images

FastAPI, REST APIs, Validation, Authentication, Database Integration, Testing,
Documentation and Deployment Concepts

1. Brainstorming and Ideation

Brainstorming defines why a FastAPI backend is needed and what problems it must solve. The team identifies users, client applications, data flows, API consumers, expected workloads, and the business or academic goals of the project. FastAPI is considered because it supports Python type hints, automatic API documentation, asynchronous programming, validation through Pydantic, and a modern development workflow. At this stage the team should explore possible endpoints, resources, authentication approaches, database choices, integrations, and deployment options without prematurely locking every implementation detail.

Detailed activities and implementation points

1. Identify the problem that the backend must solve.
2. Identify the target API consumers.
3. List the main user and system actors.
4. Identify the frontend or client applications.
5. List the resources managed by the API.
6. Brainstorm possible API endpoints.
7. Consider REST-style resource naming.
8. Identify operations for each resource.
9. Discuss authentication requirements.
10. Discuss authorization and roles.
11. Identify required third-party integrations.
12. Identify expected data volumes.
13. Consider expected request frequency.
14. Identify possible background operations.
15. Identify file upload or download needs.
16. Consider notification or messaging requirements.
17. List possible failure conditions.
18. Identify important audit events.
19. Discuss logging and monitoring needs.
20. Record assumptions and open questions.
21. Write functional API requirements.
22. Write non-functional requirements.
23. Define HTTP methods for each operation.
24. Define request body structures.
25. Define response body structures.
26. Define path and query parameters.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

1. Brainstorming and Ideation — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Define validation rules.
28. Define required and optional fields.
29. Define status codes for successful operations.
30. Define status codes for client errors.
31. Define status codes for server errors.
32. Define standard error-response structure.
33. Define authentication token behavior.
34. Define authorization rules.
35. Define pagination requirements.
36. Define filtering and sorting rules.
37. Define search requirements.
38. Define idempotency expectations where relevant.
39. Define rate-limiting expectations where needed.
40. Define performance targets.
41. Define availability expectations.
42. Create acceptance criteria for each endpoint.
43. Trace requirements to implementation modules.
44. Trace requirements to tests.
45. Choose a maintainable project structure.
46. Separate routers from business logic.
47. Separate services from database access.
48. Define Pydantic request schemas.
49. Define Pydantic response schemas.
50. Define database models.
51. Define database relationships.
52. Define dependency-injection functions.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

2. Requirement Analysis

Requirement analysis converts the backend idea into precise and testable API requirements. Functional requirements describe endpoints, resources, validation, authentication, database operations, file handling, and responses. Non-functional requirements describe performance, reliability, security, maintainability, observability, scalability, and availability. Requirements should define request and response structures, HTTP methods, status codes, error behavior, access rules, and acceptance criteria. Each major requirement should be traceable to an implementation module and a test case.

Detailed activities and implementation points

1. Record assumptions and open questions.
2. Write functional API requirements.
3. Write non-functional requirements.
4. Define HTTP methods for each operation.
5. Define request body structures.
6. Define response body structures.
7. Define path and query parameters.
8. Define validation rules.
9. Define required and optional fields.
10. Define status codes for successful operations.
11. Define status codes for client errors.
12. Define status codes for server errors.
13. Define standard error-response structure.
14. Define authentication token behavior.
15. Define authorization rules.
16. Define pagination requirements.
17. Define filtering and sorting rules.
18. Define search requirements.
19. Define idempotency expectations where relevant.
20. Define rate-limiting expectations where needed.
21. Define performance targets.
22. Define availability expectations.
23. Create acceptance criteria for each endpoint.
24. Trace requirements to implementation modules.
25. Trace requirements to tests.
26. Choose a maintainable project structure.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

2. Requirement Analysis — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Separate routers from business logic.
28. Separate services from database access.
29. Define Pydantic request schemas.
30. Define Pydantic response schemas.
31. Define database models.
32. Define database relationships.
33. Define dependency-injection functions.
34. Define configuration management.
35. Define exception-handling strategy.
36. Define authentication utilities.
37. Define authorization dependencies.
38. Define transaction boundaries.
39. Define external-service adapters.
40. Define background-task boundaries.
41. Define logging strategy.
42. Define health and readiness endpoints.
43. Define API versioning strategy if required.
44. Create a project schedule.
45. Divide the work into milestones.
46. Identify dependencies between tasks.
47. Estimate effort for each feature.
48. Plan database migration work.
49. Plan API contract reviews.
50. Plan code reviews.
51. Plan security reviews.
52. Plan automated test creation.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

3. Project Design

Project design establishes the FastAPI application's architecture. A clean design separates API routers, schemas, service logic, database access, configuration, security, and reusable utilities. The team defines resources, URL structures, request models, response models, dependency injection, transaction boundaries, and exception handling. The design should also explain how the API communicates with databases, external services, frontend applications, background tasks, and logging or monitoring systems. The goal is a backend that remains understandable as features grow.

Detailed activities and implementation points

1. Define rate-limiting expectations where needed.
2. Define performance targets.
3. Define availability expectations.
4. Create acceptance criteria for each endpoint.
5. Trace requirements to implementation modules.
6. Trace requirements to tests.
7. Choose a maintainable project structure.
8. Separate routers from business logic.
9. Separate services from database access.
10. Define Pydantic request schemas.
11. Define Pydantic response schemas.
12. Define database models.
13. Define database relationships.
14. Define dependency-injection functions.
15. Define configuration management.
16. Define exception-handling strategy.
17. Define authentication utilities.
18. Define authorization dependencies.
19. Define transaction boundaries.
20. Define external-service adapters.
21. Define background-task boundaries.
22. Define logging strategy.
23. Define health and readiness endpoints.
24. Define API versioning strategy if required.
25. Create a project schedule.
26. Divide the work into milestones.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

3. Project Design — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Identify dependencies between tasks.
28. Estimate effort for each feature.
29. Plan database migration work.
30. Plan API contract reviews.
31. Plan code reviews.
32. Plan security reviews.
33. Plan automated test creation.
34. Plan integration testing.
35. Plan performance testing.
36. Plan documentation updates.
37. Plan deployment preparation.
38. Record project risks.
39. Define mitigation actions.
40. Set milestone acceptance criteria.
41. Prepare demonstration data.
42. Define the minimum viable backend.
43. Create the Python environment.
44. Create dependency-management files.
45. Create the FastAPI application entry point.
46. Create configuration settings.
47. Create the package and module structure.
48. Create the first health endpoint.
49. Create API routers.
50. Create Pydantic schemas.
51. Create database connectivity.
52. Create database models.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

4. Project Planning

Project planning divides backend development into manageable milestones. Typical milestones include environment setup, project scaffolding, data modeling, core endpoints, authentication, validation, integrations, testing, documentation, deployment preparation, and demonstration. Dependencies and risks should be tracked. The plan should include code review, database migration work, API contract review, security checks, performance testing, and time for fixing defects. A realistic plan also identifies what belongs in the minimum viable implementation and what can be added later.

Detailed activities and implementation points

1. Define external-service adapters.
2. Define background-task boundaries.
3. Define logging strategy.
4. Define health and readiness endpoints.
5. Define API versioning strategy if required.
6. Create a project schedule.
7. Divide the work into milestones.
8. Identify dependencies between tasks.
9. Estimate effort for each feature.
10. Plan database migration work.
11. Plan API contract reviews.
12. Plan code reviews.
13. Plan security reviews.
14. Plan automated test creation.
15. Plan integration testing.
16. Plan performance testing.
17. Plan documentation updates.
18. Plan deployment preparation.
19. Record project risks.
20. Define mitigation actions.
21. Set milestone acceptance criteria.
22. Prepare demonstration data.
23. Define the minimum viable backend.
24. Create the Python environment.
25. Create dependency-management files.
26. Create the FastAPI application entry point.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

4. Project Planning — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Create configuration settings.
28. Create the package and module structure.
29. Create the first health endpoint.
30. Create API routers.
31. Create Pydantic schemas.
32. Create database connectivity.
33. Create database models.
34. Create migration configuration.
35. Implement create operations.
36. Implement read operations.
37. Implement update operations.
38. Implement delete or deactivate operations.
39. Implement pagination.
40. Implement filtering.
41. Implement sorting.
42. Implement search where required.
43. Implement validation.
44. Implement authentication.
45. Implement authorization.
46. Implement password handling where applicable.
47. Implement token handling where applicable.
48. Implement centralized exceptions.
49. Implement structured logging.
50. Implement external integrations.
51. Implement background tasks where appropriate.
52. Add environment-based configuration.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

5. Development

Development turns the FastAPI design into a working backend. The project is initialized with a clear package structure, dependency management, configuration strategy, and application entry point. Routers define API paths, Pydantic models validate input and shape output, service modules implement business logic, and database modules handle persistence. Authentication and authorization are added according to requirements. Error handling, logging, environment configuration, and automated tests should be developed alongside features instead of being postponed until the end.

Detailed activities and implementation points

1. Define mitigation actions.
2. Set milestone acceptance criteria.
3. Prepare demonstration data.
4. Define the minimum viable backend.
5. Create the Python environment.
6. Create dependency-management files.
7. Create the FastAPI application entry point.
8. Create configuration settings.
9. Create the package and module structure.
10. Create the first health endpoint.
11. Create API routers.
12. Create Pydantic schemas.
13. Create database connectivity.
14. Create database models.
15. Create migration configuration.
16. Implement create operations.
17. Implement read operations.
18. Implement update operations.
19. Implement delete or deactivate operations.
20. Implement pagination.
21. Implement filtering.
22. Implement sorting.
23. Implement search where required.
24. Implement validation.
25. Implement authentication.
26. Implement authorization.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

5. Development — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Implement password handling where applicable.
28. Implement token handling where applicable.
29. Implement centralized exceptions.
30. Implement structured logging.
31. Implement external integrations.
32. Implement background tasks where appropriate.
33. Add environment-based configuration.
34. Add CORS rules appropriate to the application.
35. Add health and readiness checks.
36. Run the application locally.
37. Inspect the generated OpenAPI documentation.
38. Write unit tests for core business logic.
39. Write API endpoint tests.
40. Test request validation.
41. Test response validation.
42. Test authentication success.
43. Test authentication failure.
44. Test authorization failure.
45. Test missing resources.
46. Test duplicate resources.
47. Test invalid identifiers.
48. Test malformed JSON or input.
49. Test database constraints.
50. Test transaction rollback behavior.
51. Test external-service failures.
52. Test pagination boundaries.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

6. Testing

Testing verifies that the FastAPI backend behaves correctly under expected and unexpected conditions. Unit tests can validate business logic and utility functions. API tests can verify routes, status codes, request validation, response schemas, authentication, authorization, and database behavior. Integration tests can exercise multiple components together. Testing should also include malformed input, missing resources, duplicate records, invalid credentials, permission failures, transaction errors, concurrency-sensitive operations, and performance-sensitive endpoints. Defects should be reproducible and retested after fixes.

Detailed activities and implementation points

1. Implement pagination.
2. Implement filtering.
3. Implement sorting.
4. Implement search where required.
5. Implement validation.
6. Implement authentication.
7. Implement authorization.
8. Implement password handling where applicable.
9. Implement token handling where applicable.
10. Implement centralized exceptions.
11. Implement structured logging.
12. Implement external integrations.
13. Implement background tasks where appropriate.
14. Add environment-based configuration.
15. Add CORS rules appropriate to the application.
16. Add health and readiness checks.
17. Run the application locally.
18. Inspect the generated OpenAPI documentation.
19. Write unit tests for core business logic.
20. Write API endpoint tests.
21. Test request validation.
22. Test response validation.
23. Test authentication success.
24. Test authentication failure.
25. Test authorization failure.
26. Test missing resources.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

6. Testing — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Test duplicate resources.
28. Test invalid identifiers.
29. Test malformed JSON or input.
30. Test database constraints.
31. Test transaction rollback behavior.
32. Test external-service failures.
33. Test pagination boundaries.
34. Test empty result sets.
35. Test large result sets.
36. Test concurrent-sensitive operations.
37. Test rate limits when implemented.
38. Test health endpoints.
39. Run the full regression suite.
40. Record reproducible defects.
41. Retest fixed defects.
42. Measure important performance paths.
43. Document project objectives.
44. Document project scope.
45. Document architecture.
46. Document folder structure.
47. Document environment setup.
48. Document required dependencies.
49. Document environment variables without exposing secrets.
50. Document database setup.
51. Document migration commands.
52. Document authentication behavior.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

7. Documentation

Documentation makes the backend understandable to developers, testers, API consumers, and future maintainers. FastAPI automatically supports interactive OpenAPI documentation, but a complete project document should also explain architecture, setup, environment variables, database configuration, authentication, endpoint behavior, error responses, testing, deployment, and maintenance. Examples of requests and responses are useful. Documentation should state assumptions, supported versions, known limitations, and operational requirements. It should be updated whenever API contracts change.

Detailed activities and implementation points

1. Write API endpoint tests.
2. Test request validation.
3. Test response validation.
4. Test authentication success.
5. Test authentication failure.
6. Test authorization failure.
7. Test missing resources.
8. Test duplicate resources.
9. Test invalid identifiers.
10. Test malformed JSON or input.
11. Test database constraints.
12. Test transaction rollback behavior.
13. Test external-service failures.
14. Test pagination boundaries.
15. Test empty result sets.
16. Test large result sets.
17. Test concurrent-sensitive operations.
18. Test rate limits when implemented.
19. Test health endpoints.
20. Run the full regression suite.
21. Record reproducible defects.
22. Retest fixed defects.
23. Measure important performance paths.
24. Document project objectives.
25. Document project scope.
26. Document architecture.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

7. Documentation — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Document folder structure.
28. Document environment setup.
29. Document required dependencies.
30. Document environment variables without exposing secrets.
31. Document database setup.
32. Document migration commands.
33. Document authentication behavior.
34. Document authorization rules.
35. Document endpoint contracts.
36. Document request examples.
37. Document response examples.
38. Document common errors.
39. Document testing commands.
40. Document deployment steps.
41. Document health checks.
42. Document logging and monitoring.
43. Document known limitations.
44. Document version history.
45. Start the demonstration with the project purpose.
46. Show the FastAPI application running.
47. Open the interactive API documentation.
48. Show the available routes.
49. Authenticate using safe demonstration credentials.
50. Create a sample resource.
51. Retrieve the created resource.
52. Update the resource.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

8. Demonstration

A backend demonstration should show a complete API workflow rather than only individual endpoints. The presenter can start the server, open the interactive API documentation, authenticate a user, create a resource, retrieve it, update it, test validation, demonstrate an error response, and delete or deactivate the resource where appropriate. The demonstration can also show database persistence, logs, health checks, and selected automated tests. A prepared dataset and predictable sequence make the demonstration easier to follow and reproduce.

Detailed activities and implementation points

1. Run the full regression suite.
2. Record reproducible defects.
3. Retest fixed defects.
4. Measure important performance paths.
5. Document project objectives.
6. Document project scope.
7. Document architecture.
8. Document folder structure.
9. Document environment setup.
10. Document required dependencies.
11. Document environment variables without exposing secrets.
12. Document database setup.
13. Document migration commands.
14. Document authentication behavior.
15. Document authorization rules.
16. Document endpoint contracts.
17. Document request examples.
18. Document response examples.
19. Document common errors.
20. Document testing commands.
21. Document deployment steps.
22. Document health checks.
23. Document logging and monitoring.
24. Document known limitations.
25. Document version history.
26. Start the demonstration with the project purpose.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

8. Demonstration — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Show the FastAPI application running.
28. Open the interactive API documentation.
29. Show the available routes.
30. Authenticate using safe demonstration credentials.
31. Create a sample resource.
32. Retrieve the created resource.
33. Update the resource.
34. Demonstrate validation failure.
35. Demonstrate an authorization failure.
36. Demonstrate a missing-resource response.
37. Show database persistence.
38. Show selected automated test results.
39. Show health or readiness status.
40. Explain the architecture.
41. Explain important security decisions.
42. Explain known limitations.
43. Create a system architecture diagram.
44. Create an API request-flow diagram.
45. Create a router-service-database diagram.
46. Create an authentication-flow diagram.
47. Create an entity-relationship diagram.
48. Create a deployment diagram.
49. Create an exception-flow diagram.
50. Create an external-integration diagram.
51. Use consistent diagram notation.
52. Use readable labels.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

9. Diagrams

Diagrams explain the FastAPI backend architecture and data flow. Useful diagrams include system architecture, API request flow, router-to-service-to-database flow, authentication flow, database entity relationships, deployment architecture, exception flow, and integration diagrams. Each diagram should communicate one main idea. Clear labels, directional arrows, consistent symbols, and logical grouping help readers understand how the backend operates. Diagrams should match the actual implementation and be updated when the architecture changes.

Detailed activities and implementation points

1. Document testing commands.
2. Document deployment steps.
3. Document health checks.
4. Document logging and monitoring.
5. Document known limitations.
6. Document version history.
7. Start the demonstration with the project purpose.
8. Show the FastAPI application running.
9. Open the interactive API documentation.
10. Show the available routes.
11. Authenticate using safe demonstration credentials.
12. Create a sample resource.
13. Retrieve the created resource.
14. Update the resource.
15. Demonstrate validation failure.
16. Demonstrate an authorization failure.
17. Demonstrate a missing-resource response.
18. Show database persistence.
19. Show selected automated test results.
20. Show health or readiness status.
21. Explain the architecture.
22. Explain important security decisions.
23. Explain known limitations.
24. Create a system architecture diagram.
25. Create an API request-flow diagram.
26. Create a router-service-database diagram.

Stage output: A reviewed, documented and testable result that can be passed into the next stage of the FastAPI backend lifecycle.

9. Diagrams — Extended Explanation

This stage should be treated as an engineering activity with clear inputs, outputs, review criteria, and evidence. For a FastAPI backend, decisions should remain traceable to API behavior, data requirements, security requirements, operational needs, and test cases. The goal is a backend that is understandable, predictable, maintainable, and ready for integration.

Extended project points

27. Create an authentication-flow diagram.
28. Create an entity-relationship diagram.
29. Create a deployment diagram.
30. Create an exception-flow diagram.
31. Create an external-integration diagram.
32. Use consistent diagram notation.
33. Use readable labels.
34. Use directional arrows.
35. Keep each diagram focused.
36. Add titles and captions.
37. Update diagrams when code architecture changes.
38. Capture the FastAPI documentation screen.
39. Capture the project folder structure.
40. Capture representative API requests.
41. Capture representative API responses.
42. Capture validation errors.
43. Capture authentication flow without revealing secrets.
44. Capture database results using safe sample data.
45. Capture automated test output.
46. Capture logs without credentials.
47. Capture health-check output.
48. Capture deployment screens where appropriate.
49. Crop irrelevant areas.
50. Keep text readable.
51. Add captions to screenshots.
52. Organize original images.

Quality focus: correctness, clear API contracts, input validation, secure access, reliable persistence, useful error handling, automated testing, observability, documentation, and maintainability.

10. Images — Visual Documentation

Images strengthen backend documentation by showing the development environment, project structure, API documentation interface, request and response examples, database tools, testing results, logs, and deployment screens. Screenshots should be readable and cropped to the relevant area. Sensitive credentials, tokens, private keys, personal data, and secrets must never be included. Each image should have a caption explaining what it demonstrates and why it is relevant to the project.

Detailed image and diagram guidelines

1. Create a system architecture diagram showing clients, FastAPI, services, database, and integrations.
2. Create an API request-flow diagram showing request, validation, business logic, persistence, and response.
3. Create a router-service-database diagram showing separation of responsibilities.
4. Create an authentication flow showing login, token issuance, verification, and protected access.
5. Create an entity-relationship diagram for important database entities.
6. Create a deployment diagram showing application, server/container, database, and network relationships.
7. Create an exception-flow diagram showing validation and server-error handling.
8. Create an integration diagram for external APIs or services.
9. Use consistent diagram symbols and terminology.
10. Use directional arrows and readable labels.
11. Give every diagram a meaningful title and caption.
12. Keep each diagram focused on one main concept.
13. Update diagrams whenever architecture changes.
14. Capture the FastAPI interactive documentation screen.
15. Capture the project folder/package structure.
16. Capture representative safe API requests.
17. Capture representative API responses.
18. Capture validation-error responses.
19. Capture authentication behavior without exposing credentials.
20. Capture safe database results.
21. Capture automated test output.
22. Capture health-check results.
23. Capture logs after removing secrets and sensitive information.
24. Capture deployment screens where appropriate.
25. Crop screenshots to the relevant area.
26. Keep screenshot text readable.
27. Add descriptive captions.
28. Use consistent image dimensions.
29. Organize original image assets.
30. Use synthetic or safe demonstration data.
31. Never include API keys, passwords, tokens, or private keys.
32. Review every screenshot before adding it to documentation.
33. Use images as evidence of implementation.
34. Keep image quality suitable for PDF viewing.
35. Check diagrams and screenshots after PDF export.
36. Store final visual assets with clear filenames.
37. Record sources for externally obtained visuals when applicable.
38. Respect image licenses and permissions.
39. Maintain a version history for important visuals.
40. Make sure visual evidence matches the documented implementation.

Integrated FastAPI Backend Workflow

Brainstorming → Requirements → Architecture & API Design → Planning → FastAPI Development → Database & Security → Testing → Documentation → Demonstration → Diagrams & Images → Deployment/Release

Final conclusion: Implementing a FastAPI backend is a complete software-engineering process. Brainstorming identifies the problem and possible API capabilities; requirement analysis makes behavior measurable; project design defines architecture, resources, schemas, dependencies, and data flow; planning organizes delivery; development implements routers, validation, services, persistence, authentication,

integrations, and error handling; testing verifies correctness and reliability; documentation explains how the API is used and maintained; demonstration shows the working system; and diagrams and images provide visual evidence. A well-engineered backend should have clear contracts, predictable validation, secure access control, maintainable modules, reliable database behavior, useful logs, automated tests, accurate documentation, and a repeatable deployment process.

Suggested final deliverables: requirements document, architecture diagram, API specification, source-code project, database schema/migrations, automated test suite, test report, API documentation, demonstration script, screenshots, deployment notes, and final project PDF.